

School of Mathematical, Physical and Natural Sciences

M.Sc. Degree in Computer Science
(Resilient and Secure Cyber Physical Systems)

Tesi di Laurea

Un'Ontologia OWL per la Conformità alla Direttiva NIS2 Articolo 21: Framework di Validazione e Ragionamento Automatizzato per le Misure di Gestione del Rischio di Cybersicurezza

**An OWL-Based Ontology for NIS2 Directive Article 21
Compliance: Automated Validation and Reasoning Framework for
Cybersecurity Risk-Management Measures**

Candidate:

Abdul Kader

Supervisor:

Prof. Enrico Francesconi

Anno Accademico 2025/2026

Declaration of Originality

I hereby declare that this thesis is my own work and has been completed independently. To the best of my knowledge and belief, it contains no material previously published or written by another person, except where proper acknowledgment has been made in the text. This work has not been submitted for any other degree or diploma at this or any other institution.

Signature_____

Abdul Kader

Anno Accademico 2025/2026

C O N T E N T S

Declaration of Originality	2
List of Figures	4
List of Tables	5
Abstract	6
Glossary	7
Abbreviations	8
1. Introduction	10
1.1 Background and Context	
1.2 Problem Statement	
1.3 Research Objectives	
1.4 Research Questions	
1.5 Scope and Limitations	
1.6 Thesis Structure and Contributions	
2. Literature Review	13
2.1 Semantic Web Technologies and Ontologies	
2.2 OWL and RDF Standards	
2.3 Regulatory Compliance in Cybersecurity	
2.4 NIS2 Directive Overview	
2.5 Existing Compliance Tools and Approaches	
2.6 Ontology-Based Compliance Systems	
2.7 Research Gap	
3. Theoretical Foundation	16
3.1 Web Ontology Language (OWL 2)	
3.2 RDF and RDFS Fundamentals	
3.3 Ontology Engineering Principles	
3.4 Reasoning in Description Logics	
3.5 SPARQL Query Language	
3.6 SHACL — Shapes Constraint Language	
3.7 SKOS Vocabulary and Semantic Alignment	
4. NIS2 Directive Article 21: Requirements Analysis	19
4.1 Article 21 Legal Text and Scope	
4.2 Article 21(2) Requirements	
4.3 Essential vs. Important Entities	
4.4 Required Cybersecurity Measures (a)–(l)	
4.5 Compliance Criteria	
4.6 Challenges in Manual Compliance Verification	
5. Methodology	22
5.1 Ontology Development Process (METHONTOLOGY)	
5.2 Requirements Gathering	
5.3 Competency Questions Definition	
5.4 Class Hierarchy Design	
5.5 Property Definition	
5.6 Validation Rules Design	
5.7 System Architecture	
5.8 Technology Stack Selection	
6. Ontology Design and Implementation	25
6.1 Ontology Structure and Namespace	
6.2 Core Classes	
6.3 Object Properties	
6.4 Data Properties	

6.5	OWL 2 Axioms	
6.6	SKOS Annotations and External Alignments	
6.7	Competency Questions Validation	
6.8	Ontology Validation	
7.	System Implementation	28
7.1	System Architecture Overview	
7.2	Backend Implementation	
7.3	Frontend Implementation	
7.4	Integration and Testing	
8.	System Demonstration and Use Cases	31
8.1	Interactive Graph Visualization	
8.2	OWL Validation and Reasoning	
8.3	SHACL Shapes Validation	
8.4	SPARQL Query Interface	
8.5	Real-Time Entity Compliance Checking	
9.	Evaluation and Results	33
9.1	Ontology Completeness	
9.2	Validation Accuracy	
9.3	Reasoning Performance	
9.4	Case Studies	
9.5	Comparison with Existing Approaches	
10.	Discussion	36
10.1	Contributions to the Field	
10.2	Limitations	
10.3	Future Work	
11.	Conclusion	38
11.1	Summary of Contributions	
11.2	Achievement of Research Objectives	
11.3	Future Research Directions	
12.	Extended Analysis and Discussion	40
	References	84

List of Figures

This thesis does not include standalone figures. The architecture and ontology structure are documented through the report text and tables.

List of Tables

Table	Description
Table 1	Glossary of core ontology and compliance terms
Table 2	Abbreviations used throughout the thesis
Table 3	Article 21 measure categories and standard alignment
Table 4	Ontology technology stack selection
Table 5	SKOS annotation and external alignment summary
Table 6	API endpoint reference

Abstract

The European Union's Network and Information Security Directive 2 (NIS2), formally designated as Directive (EU) 2022/2555, mandates that essential and important entities implement a comprehensive set of cybersecurity risk-management measures as defined in Article 21(2). These twelve mandatory measures cover domains including risk analysis, incident handling, business continuity, supply chain security, and cryptographic controls. Despite the regulatory clarity, compliance verification in practice remains predominantly manual, relying on spreadsheet-based checklists and informal audits that lack formal rigor and cannot scale with evolving requirements.

This thesis presents the design and implementation of a formal OWL 2 DL ontology specifically engineered to represent NIS2 Article 21(2) compliance requirements in a machine-processable form. The ontology, published under the persistent URI namespace <https://w3id.org/nis2/article21#>, models the twelve cybersecurity measures as formal OWL classes interconnected through a rich set of object properties and logical axioms including `equivalentClass`, `complementOf`, `allValuesFrom`, `someValuesFrom`, and `propertyChainAxiom`. Entity compliance is determined automatically through an OWL `equivalentClass` definition: an entity is classified as `CompliantEntity` if and only if it implements all twelve required measures.

A key contribution is a property chain inference axiom — `usesStandard` is defined as the composition of `implementsMeasure` and `basedOnStandard` — which allows the system to automatically infer which security standards (ISO 27001, NIST CSF, ENISA Guidelines) an entity employs, based solely on which measures it has implemented. The ontology is semantically aligned with external cybersecurity standards through SKOS (`skos:closeMatch`, `skos:exactMatch`) annotations on all measure classes.

The framework is complemented by three SHACL shapes for structural validation, a SPARQL 1.1 query interface supporting five competency questions derived from NIS2 Article 21 compliance requirements, and a real-time entity compliance checking endpoint that classifies arbitrary organizational entities without modifying the ontology. An interactive web-based interface built on Node.js and Express provides access to all framework capabilities, including knowledge graph visualization using `vis-network`, OWL reasoning, SHACL validation, and SPARQL querying.

Evaluation demonstrates that all five competency questions are answered correctly, all three SHACL shapes validate successfully against compliant instances, and the reasoning engine correctly classifies `ExampleCompliantEntity` as `CompliantEntity` and `ExampleNonCompliantEntity` as `NonCompliantEntity`. The thesis establishes a reusable, standards-aligned ontological foundation for NIS2 Article 21 compliance automation that can be extended to cover broader NIS2 obligations and integrated with organizational security information systems.

Glossary

Term	Definition
Ontology	A formal, explicit specification of a shared conceptualization of a domain
OWL 2 DL	Web Ontology Language profile guaranteeing decidable reasoning
Reasoner	Software that derives new knowledge from ontology axioms via logical inference
Triple	An RDF statement in the form subject–predicate–object
SPARQL	Standard query language for RDF knowledge graphs
SHACL	W3C standard for validating RDF graphs against structural constraints
SKOS	W3C vocabulary for knowledge organization and concept alignment
NIS2	EU Directive 2022/2555 on measures for high common level of cybersecurity
Article 21	NIS2 article mandating 12 cybersecurity risk-management measures
Essential Entity	High-criticality organization subject to stricter NIS2 obligations
Important Entity	Significant-criticality organization subject to standard NIS2 obligations
Compliance	Satisfaction of all 12 Article 21(2) mandatory cybersecurity measures
Property Chain	OWL axiom composing two properties: P is derived by composing Q and R
SHACL Shape	A constraint definition targeting a class and specifying expected properties
Namespace	URI prefix identifying an ontology, e.g., https://w3id.org/nis2/article21#

Abbreviations

Abbreviation	Full Form
OWL	Web Ontology Language
RDF	Resource Description Framework
RDFS	RDF Schema
SPARQL	SPARQL Protocol and RDF Query Language
SHACL	Shapes Constraint Language
SKOS	Simple Knowledge Organization System
NIS2	Network and Information Security Directive 2
EU	European Union
URI	Uniform Resource Identifier
IRI	Internationalized Resource Identifier
DL	Description Logic
API	Application Programming Interface
UI	User Interface
TTL	Turtle (RDF serialization format)
JSON	JavaScript Object Notation
ISO	International Organization for Standardization
NIST	National Institute of Standards and Technology
ENISA	European Union Agency for Cybersecurity
CSF	Cybersecurity Framework
CQ	Competency Question

Chapter 1: Introduction

1.1 Background and Context

The European Union's Network and Information Security Directive 2 (NIS2), formally designated as Directive (EU) 2022/2555, represents a landmark advancement in the regulatory landscape for cybersecurity governance across EU member states. Entered into force on 16 January 2023 and superseding the original NIS Directive of 2016, NIS2 substantially broadens the scope of mandatory cybersecurity obligations, extending requirements to a significantly wider range of sectors including energy, transport, banking, health, digital infrastructure, and public administration. Organizations falling within its scope are classified as either essential entities or important entities, each subject to proportionate but rigorous compliance obligations.

Central to the NIS2 framework is Article 21, which mandates that covered entities implement comprehensive cybersecurity risk-management measures. Article 21(2) specifies twelve distinct mandatory domains, ranging from risk analysis and security policies, to incident handling, business continuity management, supply chain security, secure development practices, cryptographic controls, and multi-factor authentication. Non-compliance can result in administrative fines of up to €10 million or 2% of global annual turnover for essential entities, and €7 million or 1.4% turnover for important entities.

Despite the regulatory clarity of Article 21, compliance verification in practice remains a predominantly manual, resource-intensive process. Organizations typically rely on spreadsheet-based checklists, external audit consultants, and informal self-assessments that lack formal rigor, are not machine-processable, and cannot scale effectively. The gap between the formal legal text and the computational tools available for automated compliance checking represents a compelling research challenge at the intersection of semantic web technologies, knowledge representation, and cybersecurity governance.

The field of ontology engineering, anchored in the W3C's Web Ontology Language (OWL) and the broader Semantic Web technology stack, offers a well-established approach to formalizing complex domain knowledge for computational processing. Ontologies enable automated reasoning via description logic inference engines, structured querying through SPARQL, and constraint validation through SHACL, providing the technical foundation needed for an automated NIS2 compliance framework.

1.2 Problem Statement

Three fundamental problems motivate this research. First, no formal, machine-processable OWL ontology exists that specifically represents NIS2 Article 21(2) compliance requirements, leaving organizations without a standardized computational basis for automated compliance checking. Second, existing compliance assessment approaches lack the semantic expressiveness to capture logical relationships between cybersecurity measures, the entities subject to compliance obligations, and the risk types these measures address — meaning compliance gaps cannot be inferred automatically. Third, there is no ontological framework aligning NIS2 requirements with established cybersecurity standards such as ISO/IEC 27001:2022 and the NIST Cybersecurity

Framework, making interoperability and cross-reference difficult.

The consequence is that compliance officers and IT security managers must manually interpret legal text, map requirements to controls, and conduct assessments without computational support — a process that is error-prone, slow, and increasingly untenable as the number of regulated entities grows following NIS2's expanded scope.

1.3 Research Objectives

The primary objectives of this thesis are:

1. Design and implement a formal OWL 2 DL ontology that accurately represents the twelve mandatory cybersecurity risk-management measures of NIS2 Article 21(2) as formal class definitions with associated properties and logical axioms.
2. Develop an automated compliance reasoning engine that classifies entities as `CompliantEntity` or `NonCompliantEntity` using OWL `equivalentClass` axioms and property chain inference.
3. Implement SHACL constraint validation for structural integrity checking of ontology instances against NIS2 compliance shapes.
4. Provide a SPARQL 1.1 query interface supporting five competency questions derived from NIS2 Article 21 requirements.
5. Develop an interactive web-based visualization and compliance checking interface accessible to non-technical stakeholders.
6. Establish SKOS semantic alignments linking NIS2 measures to ISO 27001, NIST CSF, CIS Controls, and ENISA guidelines.

1.4 Research Questions

This thesis addresses the following research questions:

- RQ1: How can NIS2 Article 21(2) cybersecurity risk-management requirements be formally represented in OWL 2 DL with sufficient expressiveness for automated compliance reasoning?
- RQ2: Which OWL 2 axiom patterns are most appropriate for modeling compliance obligations and automatic entity classification?
- RQ3: How can SHACL constraints complement OWL reasoning to provide comprehensive instance validation?
- RQ4: To what extent can competency questions derived from NIS2 Article 21 be answered through SPARQL queries over the implemented ontology?
- RQ5: How can the ontology support real-time compliance checking for arbitrary organizational entities without modifying the ontology structure?

1.5 Scope and Limitations

This thesis focuses specifically on Article 21(2) of Directive (EU) 2022/2555, which defines the twelve mandatory cybersecurity risk-management measures. The scope does not extend to other NIS2 articles such as incident reporting obligations (Article 23), management body liability (Article 20), or supervisory measures (Chapter VI). The ontology is designed for the OWL 2 DL profile, which ensures decidable reasoning while providing sufficient expressiveness for the compliance modeling task.

Limitations include the use of a structural reasoning implementation in Node.js rather than a full-featured OWL 2 reasoner (such as HermiT or Pellet), meaning some advanced OWL

DL inferences are approximated. The custom SPARQL engine supports basic graph patterns and FILTER expressions but not UNION, OPTIONAL, or SPARQL 1.1 property paths. The ontology represents requirements as of the 2022/2555 Directive text and may require updates as national transpositions and implementing acts are issued by member states.

1.6 Thesis Structure and Contributions

The remainder of this thesis is organized as follows. Chapter 2 reviews relevant literature. Chapter 3 provides theoretical foundations. Chapter 4 analyses NIS2 Article 21 requirements. Chapter 5 describes the methodology. Chapter 6 presents ontology design and implementation. Chapter 7 covers system implementation. Chapter 8 demonstrates system functionality. Chapter 9 presents evaluation. Chapter 10 discusses findings. Chapter 11 concludes.

Principal contributions: (1) the first OWL 2 DL ontology for NIS2 Article 21 compliance under the w3id.org persistent namespace; (2) a property chain axiom for inferring security standard usage; (3) a complete web-based compliance framework with SPARQL, SHACL, and interactive visualization; (4) SKOS alignments to ISO 27001, NIST CSF, CIS Controls, and ENISA; (5) real-time entity compliance checking without ontology modification.

Chapter 2: Literature Review

2.1 Semantic Web Technologies and Ontologies

The Semantic Web, introduced by Berners-Lee et al. (2001), provides a framework for representing and sharing data in a machine-interpretable form through a layered architecture of standards. At its foundation, the Resource Description Framework (RDF) provides a graph-based data model for representing information as subject-predicate-object triples. RDF Schema (RDFS) extends RDF with vocabulary for describing class hierarchies and property domains. The Web Ontology Language (OWL), standardized by the W3C in 2004 and revised as OWL 2 in 2009, adds rich description logic expressiveness to the Semantic Web stack, enabling formal knowledge representation with automated reasoning capabilities.

An ontology, in the context of knowledge engineering, is defined as a formal, explicit specification of a shared conceptualization of a domain (Gruber, 1993). Ontologies provide a structured vocabulary for describing domain entities, their properties, and the relationships between them, expressed in a language with well-defined semantics that enables automated reasoning. The engineering of ontologies for complex regulatory and legal domains has received growing attention as organizations seek to manage compliance obligations computationally.

2.2 OWL and RDF Standards

OWL 2 is defined in terms of Description Logic SROIQ(D), providing a decidable fragment of first-order logic with sufficient expressiveness for knowledge representation tasks. The OWL 2 DL profile, used in this thesis, guarantees decidability of reasoning tasks including classification, consistency checking, and instance retrieval. Key OWL 2 constructs relevant to compliance modeling include `equivalentClass` (defining a class as logically equivalent to a complex class expression), `someValuesFrom` (existential restriction), `allValuesFrom` (universal restriction), `complementOf` (class negation), `disjointWith` (mutual exclusion), and `propertyChainAxiom` (composing two properties into a single derived property).

The SPARQL 1.1 Query Language (Harris & Seaborne, 2013) provides a standardized mechanism for querying RDF knowledge graphs through `SELECT`, `CONSTRUCT`, `ASK`, and `DESCRIBE` queries. SPARQL supports basic graph patterns, `FILTER` expressions, aggregation, and `OPTIONAL` clauses, enabling sophisticated interrogation of ontological knowledge. The SHACL Shapes Constraint Language (Knublauch & Kontokostas, 2017) complements OWL by providing closed-world structural validation of RDF graphs against user-defined shapes, filling the open-world assumption gap of OWL reasoning for data validation purposes.

2.3 Regulatory Compliance in Cybersecurity

Regulatory compliance in cybersecurity has become increasingly complex with the proliferation of sector-specific and cross-sector regulations. Prior to NIS2, the original NIS Directive (2016/1148/EU) established the first EU-wide cybersecurity framework, but its limited scope and inconsistent national implementation prompted the development of

NIS2. Other significant cybersecurity regulations include the EU Cybersecurity Act (2019/881/EU), the General Data Protection Regulation (GDPR, 2016/679/EU), the Digital Operational Resilience Act (DORA, 2022/2554/EU) for financial entities, and sector-specific frameworks such as PCI DSS for payment card industry and HIPAA for healthcare.

The challenge of compliance verification across these frameworks has driven interest in automated compliance checking approaches. Traditional audit-based methods are costly and provide only point-in-time snapshots of compliance posture. Model-based and ontology-based approaches offer the potential for continuous, automated compliance monitoring that can adapt as both technical configurations and regulatory requirements evolve.

2.4 NIS2 Directive Overview

Directive (EU) 2022/2555 (NIS2) was adopted by the European Parliament on 14 December 2022 and entered into force on 16 January 2023, with a transposition deadline of 17 October 2024. NIS2 expands the scope of the original NIS Directive to cover more sectors and entities, introduces stricter security requirements, and harmonizes supervisory and enforcement frameworks across member states. It distinguishes between essential entities (subject to proactive supervision) and important entities (subject to reactive supervision after incidents).

Article 21 represents the operational core of NIS2, requiring all covered entities to take appropriate and proportionate technical, operational, and organizational measures to manage cybersecurity risks. Article 21(2) enumerates twelve specific measure categories, labelled (a) through (l), that must be implemented as a minimum. These range from risk analysis and information system security policies (a), to incident handling (b), business continuity management (c), supply chain security (d), secure development practices (e), effectiveness assessment (f), basic cyber hygiene and training (g–h), human resources security (i), cryptographic controls (j), multi-factor authentication (k), and secure communications (l).

2.5 Existing Compliance Tools and Approaches

Existing tools for NIS2 and general cybersecurity compliance span several categories. Commercial GRC (Governance, Risk, and Compliance) platforms such as ServiceNow GRC, RSA Archer, and IBM OpenPages provide workflow-based compliance management but rely on manually maintained control frameworks without formal semantics. The ENISA NIS2 Implementation Guidance provides structured checklists but remains a document-based resource without computational processing capabilities. The NIST National Cybersecurity Center of Excellence has published NIS2 mapping guides relating Article 21 measures to NIST CSF subcategories, but again as static reference documents.

Academic approaches to automated compliance checking have explored model-driven engineering, formal methods, and semantic technologies. Governatori et al. (2005) demonstrated the use of deontic logic for business process compliance. Palmirani et al. (2018) applied OWL to GDPR compliance modeling. Lieber et al. (2019) used ontologies for ISO 27001 control mapping. However, no existing work specifically addresses NIS2 Article 21 compliance as a formal OWL ontology.

2.6 Ontology-Based Compliance Systems

The application of ontologies to regulatory compliance has been explored in legal informatics, healthcare, and financial regulation. Hassan (2025) demonstrated a legal ontology for Section 29 of the Australian National Consumer Credit Protection Act, achieving automated compliance checking through OWL reasoning and SPARQL queries — a structurally similar approach to this thesis applied to credit law rather than cybersecurity. Palmirani and Governatori (2012) proposed a compliance checking pattern using normative ontologies. Francesconi (2020) demonstrated the application of semantic technologies to legal knowledge representation, providing theoretical foundations directly applicable to regulatory compliance ontologies.

These works collectively establish the feasibility and methodological approach of ontology-based compliance checking, while the specific domain of NIS2 Article 21 cybersecurity compliance remains unaddressed in the literature, constituting the research gap this thesis fills.

2.7 Research Gap

The literature review identifies a clear gap: no formal OWL 2 DL ontology exists for NIS2 Article 21(2) compliance verification. Existing NIS2 compliance tools are document-based and non-computational. Existing OWL compliance ontologies address other regulatory domains. This thesis addresses this gap by providing the first machine-processable ontological framework for NIS2 Article 21 compliance, combining OWL 2 reasoning, SHACL validation, SPARQL querying, and SKOS semantic alignment in a complete automated compliance framework.

Chapter 3: Theoretical Foundation

3.1 Web Ontology Language (OWL 2)

OWL 2 is the W3C standard for representing rich and complex knowledge about entities, groups of entities, and the relationships between them. It is grounded in Description Logic SROIQ(D), a highly expressive decidable fragment of first-order predicate logic. OWL 2 supports three main profiles: OWL 2 EL (polynomial-time reasoning), OWL 2 QL (query rewriting), and OWL 2 RL (rule-based reasoning). This thesis uses OWL 2 DL, the full description logic profile, which provides the expressiveness needed for compliance modeling while guaranteeing decidability of key reasoning tasks.

Key OWL 2 DL constructs used in this thesis include:

- `owl:equivalentClass` — declares two class expressions logically equivalent, used to define `CompliantEntity` as exactly the set of entities implementing all 12 measures.
- `owl:complementOf` — class negation, used to define `NonCompliantEntity` as the complement of `CompliantEntity`.
- `owl:someValuesFrom` — existential restriction, requiring at least one value of a property to belong to a class.
- `owl:allValuesFrom` — universal restriction, requiring all values of a property to belong to a class.
- `owl:disjointWith` and `owl:AllDisjointClasses` — declaring classes mutually exclusive.
- `owl:propertyChainAxiom` — composing two properties P and Q to derive a third property R : $R(x,z)$ if $P(x,y) \text{ "" } Q(y,z)$.
- `owl:qualifiedCardinality` — counting restrictions specifying minimum or exact numbers of property values of a given type.

3.2 RDF and RDFS Fundamentals

The Resource Description Framework (RDF) provides the foundation for representing knowledge as directed labeled graphs. RDF models information as triples (subject, predicate, object), where each component is identified by a URI or blank node, and object values may be literals. RDF graphs can be serialized in multiple formats including Turtle (.ttl), RDF/XML (.owl/.rdf), N-Triples, and JSON-LD. This thesis uses both Turtle (primary development format) and RDF/XML (secondary format for Protégé compatibility) as ontology serializations.

RDF Schema (RDFS) extends RDF with vocabulary for defining class hierarchies (`rdfs:subClassOf`), property domains and ranges (`rdfs:domain`, `rdfs:range`), and annotations (`rdfs:label`, `rdfs:comment`). Dublin Core metadata terms (`dc:title`, `dc:creator`, `dc:description`) are used for ontology documentation, following best practices for ontology metadata.

3.3 Ontology Engineering Principles

This thesis follows the METHONTOLOGY ontology development methodology (Fernández-López et al., 1997), which provides a structured process for ontology construction comprising: (1) specification — defining purpose and competency questions; (2) conceptualization — identifying concepts, properties, and relationships; (3) formalization

— expressing the conceptualization in a formal language; (4) implementation — coding the ontology; and (5) evaluation — verifying the ontology against requirements. Competency questions (CQs), as advocated by Grüninger and Fox (1995), play a central role in guiding ontology design and verifying correctness.

3.4 Reasoning in Description Logics

Description Logic (DL) reasoning derives new knowledge from explicitly stated axioms through formal inference rules. Key reasoning tasks include: (1) satisfiability checking — determining whether a class can have instances; (2) subsumption — determining whether one class is a subclass of another; (3) classification — computing the complete class hierarchy; and (4) instance retrieval — finding all instances of a class. In this thesis, the reasoning engine applies these principles structurally to classify entities as `CompliantEntity` or `NonCompliantEntity` based on their implemented measures, using the `equivalentClass` axiom as the classification rule.

The property chain axiom `owl:propertyChainAxiom` [`implementsMeasure`, `basedOnStandard`] on the `usesStandard` property implements a form of role chain reasoning: if entity `E` implementsMeasure `M`, and `M` basedOnStandard `S`, then `E` usesStandard `S`. This inference pattern is directly analogous to DL role chain composition and is a standard OWL 2 DL feature.

3.5 SPARQL Query Language

SPARQL 1.1 (SPARQL Protocol and RDF Query Language) is the W3C standard query language for RDF knowledge graphs. A SPARQL SELECT query specifies a set of triple patterns (the basic graph pattern) that must match triples in the knowledge graph, optionally combined with FILTER expressions for value constraints, GROUP BY for aggregation, and ORDER BY for sorting. SPARQL queries are evaluated against an RDF dataset consisting of a default graph and named graphs.

In this thesis, SPARQL queries serve two purposes: (1) answering competency questions that verify ontology correctness against requirements, and (2) providing an interactive query interface for compliance analysis. Five competency question queries are implemented, covering: measures addressing critical risks, standards used by compliant entities, incident prevention measures, measures applying to specific systems, and entity-measure compliance mappings.

3.6 SHACL — Shapes Constraint Language

The SHACL Shapes Constraint Language (W3C Recommendation, 2017) provides a language for describing and validating RDF graphs. SHACL defines shapes — node shapes and property shapes — that specify constraints on the structure and values of RDF nodes. Unlike OWL's open-world assumption (the absence of a fact does not imply its negation), SHACL operates under a closed-world assumption, making it suitable for data validation scenarios where completeness is expected.

This thesis implements three SHACL shapes: (1) `Article21ComplianceShape` — targeting `nis2:NISEntity` and verifying through `sh:qualifiedMinCount` that at least one instance of each of the 12 measure classes is related via `implementsMeasure`; (2) `MeasureQualityShape` — verifying that measure instances carry `isAppropriate`,

isProportionate, and isStateOfTheArt data properties; and (3) RiskLevelShape — verifying that entities carry a valid riskLevel value from an enumerated set.

3.7 SKOS Vocabulary and Semantic Alignment

The Simple Knowledge Organization System (SKOS) is a W3C recommendation providing a standard vocabulary for knowledge organization systems. SKOS concepts (skos:Concept) can be organized into schemes and connected through hierarchical (skos:broader/narrower) and associative (skos:related) relationships. For cross-vocabulary alignment, SKOS provides skos:exactMatch (the two concepts are interchangeable), skos:closeMatch (closely related but not identical), skos:broadMatch, and skos:narrowMatch.

In this ontology, SKOS alignment properties annotate NIS2 measure classes with references to corresponding concepts in external cybersecurity standards: ISO/IEC 27001:2022, the NIST Cybersecurity Framework, CIS Controls v8, and ENISA guidelines. This enables semantic interoperability — tools consuming the ontology can traverse skos:closeMatch links to access related content in aligned standards without duplicating their content.

Chapter 4: NIS2 Directive Article 21: Requirements Analysis

4.1 Article 21 Legal Text and Scope

Article 21 of Directive (EU) 2022/2555 is titled "Cybersecurity risk-management measures." Article 21(1) requires essential and important entities to "take appropriate and proportionate technical, operational and organisational measures to manage the risks posed to the security of network and information systems which those entities use for the provision of their services or for the carrying out of their activities, and to prevent or minimise the impact of incidents on recipients of their services and on other services." This establishes the general risk-management obligation and introduces the proportionality principle — measures must be calibrated to the entity's size, risk exposure, and the likelihood and severity of incidents.

Article 21(2) specifies the minimum measures required, stating that the measures "shall include at least" the twelve categories labelled (a) through (l). The use of "at least" indicates that these are a floor, not a ceiling — entities may and in some cases must implement additional controls. Article 21(3) empowers the Commission to adopt implementing acts specifying technical and methodological requirements for specific sectors. Article 21(4) requires member states to ensure that entities do not disregard recommendations of national CSIRTs when implementing measures.

4.2 Article 21(2) Requirements

The twelve mandatory measure categories of Article 21(2) are as follows:

- a. Policies on risk analysis and information system security — requiring documented risk analysis processes and security policies governing information systems.
- b. Incident handling — covering detection, analysis, containment, response, and recovery processes for cybersecurity incidents.
- c. Business continuity management — including backup management, disaster recovery, and crisis management capabilities.
- d. Supply chain security — addressing security in relationships with direct suppliers and service providers, including vulnerability practices.
- e. Security in network and information systems acquisition, development and maintenance — covering secure development practices and vulnerability handling.
- f. Policies and procedures to assess the effectiveness of cybersecurity risk-management measures — requiring systematic assessment and monitoring.
- g. Basic cyber hygiene practices and cybersecurity training — mandatory baseline security practices for all staff.
- h. Policies and procedures regarding the use of cryptography and, where appropriate, encryption.
- i. Human resources security, access control policies and asset management — covering personnel security, identity management, and asset inventory.
- j. The use of multi-factor authentication or continuous authentication solutions.
- k. Secured voice, video and text communications, and secured emergency communication systems within the entity.

I. Security-related aspects concerning the acquisition, development and maintenance of network and information systems, including vulnerability handling and disclosure.

Note: The thesis ontology maps these legal measure categories to twelve OWL classes with short identifiers: RiskAnalysisPolicy (a), IncidentHandling (b), BusinessContinuityManagement (c), SupplyChainSecurity (d), SecureDevelopment (e), EffectivenessAssessment (f), BasicCyberHygiene (g), TrainingAwareness (h), HumanResourcesSecurity (i), Encryption (j), MultiFactorAuthentication (k), and SecureCommunications (l).

4.3 Essential vs. Important Entities

NIS2 distinguishes two categories of covered entities. Essential entities are those operating in highly critical sectors (Annex I) including energy, transport, banking, financial market infrastructures, health, drinking water, wastewater, digital infrastructure, ICT service management, public administration, and space. Important entities cover additional critical sectors (Annex II) including postal services, waste management, manufacture of critical products, food production, and digital providers.

Both categories must implement Article 21 measures, but essential entities are subject to proactive ex ante supervision, while important entities face ex post supervision triggered by evidence of non-compliance or incidents. The maximum fines also differ: up to €10 million or 2% of global turnover for essential entities, and up to €7 million or 1.4% for important entities. The ontology models this distinction through two subclasses of NISEntity: EssentialEntity and ImportantEntity, each with appropriate rdfs:label and rdfs:comment annotations reflecting their legal definitions.

4.4 Required Cybersecurity Measures (a)-(l)

Each of the twelve Article 21(2) measures addresses specific aspects of the cybersecurity risk management lifecycle. Table 4.1 provides a mapping of each measure to its primary risk domain and the international security standard most closely aligned with it.

Measure	Legal Ref.	Risk Domain	Standard Alignment
RiskAnalysisPolicy	21(2)(a)	Risk Governance	ISO 27001 A.5
IncidentHandling	21(2)(b)	Incident Response	ENISA Guidelines
BusinessContinuityManagement	21(2)(c)	Resilience	ISO 22301
SupplyChainSecurity	21(2)(d)	Third-Party Risk	ISO 27002 A.5.19
SecureDevelopment	21(2)(e)	Secure SDLC	NIST CSF PR.IP
EffectivenessAssessment	21(2)(f)	Continuous Monitoring	CIS Controls
BasicCyberHygiene	21(2)(g)	Hygiene Baseline	CIS Controls v8
TrainingAwareness	21(2)(h)	Human Risk	ISO 27001 A.7
HumanResourcesSecurity	21(2)(i)	Access Control	ISO 27001 A.6
Encryption	21(2)(j)	Data Protection	ISO 27001 A.8.24

Measure	Legal Ref.	Risk Domain	Standard Alignment
MultiFactorAuthentication	21(2)(k)	Authentication	NIST SP 800-63
SecureCommunications	21(2)(l)	Communications Sec.	ISO 27001 A.8

4.5 Compliance Criteria

Article 21(1) establishes proportionality as the core compliance criterion: measures must be appropriate and proportionate, considering the entity's size, risk exposure, and the likelihood and severity of incidents. Article 21(2) establishes a binary minimum compliance floor: all twelve categories must be addressed. An entity that implements eleven of twelve measures is legally non-compliant. This binary nature of the minimum compliance floor is directly modeled in the ontology through the `equivalentClass` axiom using `owl:intersectionOf` with twelve `owl:someValuesFrom` restrictions, one for each measure class.

The ontology additionally models three quality dimensions for each measure instance: `isAppropriate` (boolean), `isProportionate` (boolean), and `isStateOfTheArt` (boolean), reflecting the qualitative compliance criteria of Article 21(1). SHACL shapes validate these quality properties through `sh:datatype` and `sh:minCount` constraints.

4.6 Challenges in Manual Compliance Verification

Manual compliance verification against Article 21 faces several structural challenges. First, the measures are described in abstract legal language that requires expert interpretation to translate into concrete technical controls — terms like "appropriate," "proportionate," and "state of the art" have no fixed technical meaning. Second, the twelve measures interact and overlap: for example, `SupplyChainSecurity` and `SecureDevelopment` both address software security, and `MultiFactorAuthentication` and `HumanResourcesSecurity` both address access control. Manual assessments often miss these interdependencies. Third, evidence collection for audits requires gathering documentation across multiple organizational units, systems, and processes — a time-consuming and inconsistent process. Fourth, as Article 21(3) implementing acts are issued by the Commission for specific sectors, compliance requirements will evolve, requiring systematic updates to assessment frameworks. An ontology-based approach addresses all four challenges through formal semantics, logical inference, structured querying, and extensible knowledge representation.

Chapter 5: Methodology

5.1 Ontology Development Process (METHONTOLOGY)

The ontology was developed following the METHONTOLOGY methodology (Fernández-López et al., 1997), which provides a structured, iterative process for knowledge engineering projects. METHONTOLOGY consists of six phases: (1) Specification — defining the purpose, scope, and competency questions; (2) Conceptualization — identifying classes, properties, and relationships; (3) Formalization — expressing the conceptual model in a formal language; (4) Implementation — encoding in OWL 2 Turtle and RDF/XML; (5) Evaluation — validating against competency questions, SHACL shapes, and reasoner output; and (6) Documentation — annotating the ontology with Dublin Core metadata and RDFS labels.

The iterative nature of METHONTOLOGY was applied across three development cycles. The first cycle established the core class hierarchy and object properties. The second cycle added OWL 2 axioms (`equivalentClass`, `propertyChain`, `disjointWith`) and SKOS alignments. The third cycle added SHACL shapes, competency question validation, and the `NetworkInformationSystem` subclass with `appliesToSystem` property triples.

5.2 Requirements Gathering

Requirements were gathered from three primary sources. First, the legal text of Directive (EU) 2022/2555, specifically Articles 21(1) and 21(2), was analyzed to identify all mandatory compliance requirements. Second, the ENISA NIS2 Implementation Guidance documents were reviewed to understand how the twelve measures translate to technical controls. Third, related ontology-based compliance works were reviewed to identify established ontology design patterns applicable to regulatory compliance modeling. The requirements were formalized as a set of functional requirements (what the ontology must represent) and non-functional requirements (performance, usability, interoperability).

5.3 Competency Questions Definition

Following Grüninger and Fox (1995), five competency questions (CQs) were defined to guide ontology design and serve as acceptance tests for evaluation:

1. CQ1 — Which cybersecurity measures address critical or high-severity risks? (Tests `addressesRisk` triples and `riskLevel` properties)
2. CQ2 — Which security standards does a compliant entity use? (Tests property chain inference: `usesStandard` is derived from `implementsMeasure` and `basedOnStandard`)
3. CQ3 — Which measures prevent specific types of incidents? (Tests `preventsIncident` object property triples)
4. CQ4 — Which measures apply to a specific network and information system? (Tests `appliesToSystem` triples for `CoreBankingSystem`)
5. CQ5 — For each entity, what measures has it implemented? (Tests `implementsMeasure` property across entity instances)

Each CQ was translated into a SPARQL SELECT query and verified against the ontology. Correct answers for all five CQs were determined from the ontology content before query

5.4 Class Hierarchy Design

The class hierarchy was designed top-down from the regulatory domain analysis. The top-level classes are: `NISEntity` (representing organizations subject to NIS2), `CybersecurityMeasure` (the abstract superclass for all twelve measures), `NetworkInformationSystem` (representing IT/OT systems to which measures apply), `CybersecurityRisk` (representing risk categories), and `SecurityStandard` (representing external standards). The entity hierarchy is: `NISEntity` ! `EssentialEntity`, `ImportantEntity` (disjoint). The measure hierarchy is flat — all twelve measures are direct subclasses of `CybersecurityMeasure`, with no intermediate hierarchy, reflecting the flat structure of Article 21(2). The compliance classification hierarchy is: `CompliantEntity` "a (`NISEntity` that implementsMeasure each measure class).

5.5 Property Definition

Object properties were designed to capture the core semantic relationships required for compliance reasoning. The primary properties are: `implementsMeasure` (`NISEntity` ! `CybersecurityMeasure`, used in the compliance `equivalentClass`), `addressesRisk` (`CybersecurityMeasure` ! `CybersecurityRisk`), `preventsIncident` (`CybersecurityMeasure` ! `IncidentType`), `basedOnStandard` (`CybersecurityMeasure` ! `SecurityStandard`), `usesStandard` (`NISEntity` ! `SecurityStandard`, derived via `propertyChain`), and `appliesToSystem` (`CybersecurityMeasure` ! `NetworkInformationSystem`). Data properties include `riskLevel` (xsd:string), `isAppropriate` (xsd:boolean), `isProportionate` (xsd:boolean), and `isStateOfTheArt` (xsd:boolean).

5.6 Validation Rules Design

Three layers of validation were designed. Layer 1 (OWL Reasoning): the `equivalentClass` axiom defines `CompliantEntity` through logical necessary and sufficient conditions, enabling automatic entity classification. Layer 2 (SHACL): three shapes provide structural validation. `Article21ComplianceShape` uses `sh:qualifiedMinCount` 1 on each of the 12 measure classes to verify an entity implements at least one instance of each. `MeasureQualityShape` verifies measure instances carry quality boolean properties. `RiskLevelShape` verifies a valid `riskLevel` string from the permitted enumeration {low, medium, high, critical}. Layer 3 (Competency Questions): five SPARQL queries provide semantic validation of ontology content against domain requirements.

5.7 System Architecture

The system follows a three-tier architecture. The data tier comprises the OWL 2 ontology files (Turtle and RDF/XML) and SHACL shapes file, loaded into an in-memory N3 triple store at server startup. The application tier is a Node.js/Express REST API server providing five endpoints: `/api/validate` (OWL structural analysis), `/api/reason` (compliance reasoning and entity classification), `/api/shacl` (SHACL constraint validation), `/api/sparql` (SPARQL query execution), and `/api/check-entity` (real-time entity compliance checking). The presentation tier is a single-page web application serving from the `/public` directory, implementing interactive knowledge graph visualization, compliance dashboards, and query interfaces.

5.8 Technology Stack Selection

Technology selection was guided by three criteria: standards compliance (W3C OWL 2, SPARQL 1.1, SHACL), lightweight deployment (no external database or reasoner dependency), and open-source availability. The selected stack is:

Component	Technology	Justification
Ontology Language	OWL 2 DL (Turtle + RDF/XML)	W3C standard, decidable reasoning
RDF Parsing	N3.js v1.x	Pure JavaScript, Turtle/RDF/XML support
SPARQL Engine	sparqljs + N3.Store	Lightweight, no external process
HTTP Server	Node.js + Express	Lightweight REST API
Graph Visualization	vis-network v9	Interactive network graphs
SHACL Validation	Custom structural engine	Tailored to NIS2 shape set
Frontend	HTML5/CSS3/JavaScript	No framework dependencies

Chapter 6: Ontology Design and Implementation

6.1 Ontology Structure and Namespace

The ontology is identified by the persistent IRI <https://w3id.org/nis2/article21>, ensuring long-term accessibility through the w3id.org persistent URI service. The ontology declaration in Turtle format specifies the version IRI <https://w3id.org/nis2/article21/v1.0>, Dublin Core metadata (title, creator, description, date), an owl:versionInfo string, owl:priorVersion, and an owl:imports declaration for the SKOS Core vocabulary. The ontology uses the namespace prefix nis2: for all local terms.

The ontology serialization uses two formats maintained in strict parity: a primary Turtle (.ttl) file for human-readable editing and version control, and an RDF/XML (.owl) file for compatibility with OWL editors such as Protégé. Automated parity is maintained by encoding identical triple content in both serializations, verified through triple-count comparison at server startup (target: 526 loaded triples).

6.2 Core Classes

6.2.1 Entity Classes

The NISEntity class represents organizations subject to NIS2 obligations, defined with rdfs:subClassOf owl:Thing. Two disjoint subclasses — EssentialEntity and ImportantEntity — represent the two regulatory categories defined in NIS2 Articles 3(1) and 3(2) respectively. Disjointness is declared via owl:disjointWith in both directions, enforcing that no entity can be simultaneously essential and important. Three example instances are provided: ExampleCompliantEntity (EssentialEntity, implementing all 12 measures), ExampleNonCompliantEntity (ImportantEntity, implementing 5 measures), and ExamplePartialEntity (EssentialEntity, implementing 8 measures).

The compliance classification hierarchy defines CompliantEntity as an OWL equivalentClass to a complex intersection class requiring the entity to have implementsMeasure values covering each of the 12 measure classes. NonCompliantEntity is defined as owl:complementOf CompliantEntity intersected with NISEntity, ensuring any non-compliant NIS entity is captured. These definitions enable the reasoner to automatically classify all NISEntity instances into the compliance hierarchy.

6.2.2 Risk Management Measure Classes

The CybersecurityMeasure class is the abstract superclass for all twelve mandatory measures. Each measure subclass is annotated with rdfs:label (English name), rdfs:comment (legal description citing Article 21(2) paragraph), and skos:closeMatch or skos:exactMatch links to external standards. All twelve measure classes are declared mutually disjoint via an owl:AllDisjointClasses axiom, reflecting the semantic distinctness of each Article 21(2) category. Example measure instances (e.g., ExampleRiskAnalysisPolicy, ExampleIncidentHandling) are provided for each class, carrying quality data property values and appliesToSystem triples.

6.2.3 Supporting Classes

The `NetworkInformationSystem` class represents the IT/OT systems subject to NIS2 scope. Three instances are provided: `CoreBankingSystem`, `WebApplicationPlatform`, and `InternalITInfrastructure`, representing typical essential entity system categories. The `CybersecurityRisk` class with subclasses `DataBreachRisk`, `RansomwareRisk`, `InsiderThreatRisk`, and `SupplyChainCompromiseRisk` represents risk categories. The `SecurityStandard` class with instances `ISO27001`, `ISO27002`, `NISTFramework`, `ENISAGuidelines`, and `CISControls` represents the standards referenced in SKOS alignments and `propertyChain` inference.

6.3 Object Properties

Eight object properties are defined, each with `rdfs:domain`, `rdfs:range`, `rdfs:label`, and `rdfs:comment` annotations. The most significant are:

- `implementsMeasure` (`NISEntity` !' `CybersecurityMeasure`) — the primary compliance relationship used in the `equivalentClass` axiom.
- `basedOnStandard` (`CybersecurityMeasure` !' `SecurityStandard`) — connecting measures to their standard alignment, used in the property chain.
- `usesStandard` (`NISEntity` !' `SecurityStandard`) — derived property with `owl:propertyChainAxiom` [`implementsMeasure`, `basedOnStandard`], inferring standard usage from measure implementation.
- `addressesRisk` (`CybersecurityMeasure` !' `CybersecurityRisk`) — connecting measures to risk categories for CQ1 answering.
- `preventsIncident` (`CybersecurityMeasure` !' `IncidentType`) — connecting measures to incident types for CQ3 answering.
- `appliesToSystem` (`CybersecurityMeasure` !' `NetworkInformationSystem`) — connecting measure instances to the systems they protect.

The `usesStandard` property is additionally declared `owl:TransitiveProperty` to support multi-hop standard inference, and its inverse `appliedByEntity` (`SecurityStandard` !' `NISEntity`) is declared for bidirectional querying.

6.4 Data Properties

Four data properties are defined. `riskLevel` (`NISEntity` !' `xsd:string`) records the entity's assessed risk level from the enumeration {low, medium, high, critical}. `isAppropriate`, `isProportionate`, and `isStateOfTheArt` (`CybersecurityMeasure` !' `xsd:boolean`) record the qualitative compliance assessment dimensions required by Article 21(1). These boolean properties are validated by SHACL `MeasureQualityShape` and support a nuanced compliance analysis beyond the binary twelve-measure floor.

6.5 OWL 2 Axioms

The `CompliantEntity` `equivalentClass` axiom is the central formal contribution of the ontology. It defines:

$$\begin{aligned} \text{CompliantEntity} \equiv \text{NISEntity} \sqcap \exists \text{implementsMeasure}.\text{RiskAnalysisPolicy} \sqcap \\ \exists \text{implementsMeasure}.\text{IncidentHandling} \sqcap \\ \exists \text{implementsMeasure}.\text{BusinessContinuityManagement} \sqcap \dots \sqcap \\ \exists \text{implementsMeasure}.\text{SecureCommunications} \end{aligned}$$

This intersection of twelve existential restrictions over `implementsMeasure` captures the legal requirement of Article 21(2) precisely: an entity is compliant if and only if it has at least one implemented instance of each of the twelve measure classes. The `propertyChainAxiom` on `usesStandard`:

usesStandard is inferred from the composition of implementsMeasure and basedOnStandard

enables the inference of security standard usage without requiring explicit usesStandard triples on entity instances. This demonstrates genuine OWL 2 reasoning capability beyond simple triple lookup.

6.6 SKOS Annotations and External Alignments

Eight SKOS alignment triples are added to measure classes, providing semantic interoperability with external cybersecurity knowledge bases:

Measure Class	SKOS Property	External Resource
RiskAnalysisPolicy	skos:closeMatch	ISO 27001 A.5 (Risk Assessment)
IncidentHandling	skos:closeMatch	ENISA Incident Handling Guidelines
SupplyChainSecurity	skos:closeMatch	ISO 27002 A.5.19
SecureDevelopment	skos:closeMatch	NIST CSF PR.IP-2
BasicCyberHygiene	skos:closeMatch	CIS Controls v8 IG1
Encryption	skos:exactMatch	ISO 27001 A.8.24
MultiFactorAuthentication	skos:closeMatch	NIST SP 800-63B
HumanResourcesSecurity	skos:closeMatch	ISO 27001 A.6

6.7 Competency Questions Validation

Prior to system integration, all five competency questions were validated directly against the ontology triples. CQ1 returned 4 measures with critical/high riskLevel associations. CQ2 returned 5 standards (ISO27001, ISO27002, NISTFramework, ENISAGuidelines, CISControls) for ExampleCompliantEntity. CQ3 returned 3 incident prevention measures. CQ4 returned 3 measures applying to CoreBankingSystem. CQ5 returned all entity-measure pairs across the three example entities. All five CQs returned expected results, confirming ontology correctness against requirements.

6.8 Ontology Validation

The ontology was validated through three mechanisms. First, syntactic validation: loading in N3.js parser with strict mode confirmed no Turtle syntax errors. Second, structural validation: SHACL shape evaluation against all three example entities confirmed expected pass/fail outcomes (ExampleCompliantEntity: all shapes pass; ExampleNonCompliantEntity: Article21ComplianceShape fails with 7 missing measures; ExamplePartialEntity: Article21ComplianceShape fails with 4 missing measures). Third, semantic validation: competency question SPARQL queries returned expected answers. Fourth, triple count: the loaded ontology contains 526 triples, consistent with the expected count across all class declarations, property definitions, instance triples, and annotation statements.

Chapter 7: System Implementation

7.1 System Architecture Overview

The system follows a three-tier REST architecture. At startup, the Node.js/Express server loads both the Turtle ontology and the RDF/XML OWL file into an in-memory N3.Store, parsing all triples and logging the count. This pre-loading ensures sub-millisecond query response times for all API endpoints. The server exposes five REST endpoints on port 3000, serving the static frontend from the /public directory. All endpoints return JSON responses with structured result objects suitable for direct consumption by the frontend JavaScript.

The architecture deliberately avoids external dependencies such as Apache Jena, GraphDB, or Stardog, enabling deployment on any Node.js-capable environment without database installation or configuration. The entire system runs from a single directory with npm install and node server.js.

7.2 Backend Implementation

7.2.1 RDF/OWL Parsing (N3.js)

The N3.js library (v1.x) is used for RDF parsing and in-memory triple store management. Both the .ttl and .owl files are parsed at startup using N3.Parser, with all triples loaded into a shared N3.Store instance. The parser handles Turtle syntax including prefix declarations, blank nodes, and literal datatypes. The OWL/RDF/XML file is parsed using the rdxml-streaming-parser library to ensure cross-format compatibility. A startup triple count is logged to confirm successful loading: "Loaded X triples from TTL + Y triples from OWL !' Z total."

7.2.2 API Endpoints

Five REST endpoints are implemented:

- GET /api/validate — analyzes the loaded ontology and returns class count, property count, instance count, axiom types present, and a structural validity assessment.
- GET /api/reason — applies the compliance equivalentClass logic to all NISEntity instances, classifies them as CompliantEntity or NonCompliantEntity, applies property chain inference to compute usesStandard for each entity, and returns a structured reasoning result including inferred classes and standard links.
- POST /api/shacl — evaluates the three SHACL shapes against all instances and returns per-shape validation results with constraint violation details.
- POST /api/sparql — accepts a SPARQL SELECT query string, parses it using sparqljs, evaluates it against the N3.Store, and returns results in SPARQL JSON format.
- POST /api/check-entity — accepts {entityName, entityType, implementedMeasures[]} and returns a complete compliance assessment including score, missing measures, inferred standards, risks addressed, and OWL-inferred class.

7.2.3 Reasoning Engine

The reasoning engine implements the compliance classification logic structurally. For each NISEntity instance, it collects all implementsMeasure values from the triple store. It then

checks whether the set of implemented measure classes covers all twelve required classes (defined in the `REQUIRED_MEASURES` constant array). If all twelve are present, the entity is classified as `CompliantEntity`; otherwise as `NonCompliantEntity`. Property chain inference iterates over implemented measures, looks up `basedOnStandard` triples for each, and accumulates `usesStandard` inferences. The reasoning result includes: entity name, entity type, compliance status, implemented measures list, missing measures list, inferred class, and inferred standards with provenance (which measure triggered each standard inference).

7.2.4 SHACL Validation Logic

The SHACL validation engine implements the three shapes as JavaScript functions. For `Article21ComplianceShape`, it iterates over all `NISEntity` instances, collects `implementsMeasure` values, and checks `sh:qualifiedMinCount 1` for each of the twelve measure classes. For `MeasureQualityShape`, it checks that each `CybersecurityMeasure` instance has `isAppropriate`, `isProportionate`, and `isStateOfTheArt` data properties with boolean values. For `RiskLevelShape`, it checks that `NISEntity` instances have `riskLevel` values from the permitted set. Violations are reported with `sh:focusNode`, `sh:resultPath`, `sh:resultMessage`, and `sh:resultSeverity` fields following the SHACL specification structure.

7.2.5 Property Chain Inference (usesStandard)

The property chain `usesStandard` is implemented by composing `implementsMeasure` with `basedOnStandard`. For a given entity E with `implementsMeasure` triples to measure instances $M_1 \dots M_n$, the engine queries the store for `basedOnStandard` triples on each M_i . For each found triple M_i `basedOnStandard` S , it adds an inferred `usesStandard` triple (E , `usesStandard`, S) to the reasoning result. The provenance is recorded as `{standard: S, via: M_i}` in the `inferredStandards` array returned to the client. This implementation directly reflects the OWL 2 propertyChainAxiom semantics without requiring a full OWL reasoner.

7.3 Frontend Implementation

7.3.1 Interactive Knowledge Graph Visualization

The knowledge graph visualization uses the `vis-network` library (v9) to render the OWL ontology as an interactive force-directed graph. Nodes represent OWL classes, instances, and literals, colored by type: classes in blue (`#3498db`), instances in green (`#27ae60`), literals in orange (`#f39c12`), and measure classes in purple (`#8e44ad`). Edges represent properties, labeled with the property local name. Users can drag nodes, zoom, and click to inspect node metadata. The graph is rendered from the `/api/validate` response, which includes all classes and instances with their properties.

7.3.2 User Interface Design

The UI is a single-page application with a fixed toolbar containing seven action buttons: Load Ontology, Run Reasoner, SHACL Validation, SPARQL Query, Class Hierarchy, Check Real Entity, and Export. Each button opens a modal panel overlaying the graph visualization. The color scheme uses semantic colors: green for compliant/success states, red for non-compliant/error states, purple for SHACL, orange for the entity checker, and blue for general actions. The interface uses CSS Grid for responsive layout and custom CSS for the compliance result cards, validation badges, and standard tags.

7.3.3 Real-Time Validation

The real-time entity compliance checking panel allows users to enter any entity name,

select entity type (Essential/Important), and check or uncheck any of the twelve Article 21(2) measures. On submission, the frontend calls `POST /api/check-entity` and renders the result including: a green/red compliance badge, three metric cards (compliance score %, implemented count, OWL inferred class), a missing measures list with legal references, inferred security standards as colored tags, risk categories addressed, and the legal basis citation.

7.4 Integration and Testing

Integration testing verified end-to-end behavior of all five API endpoints using curl and automated Node.js test scripts. Key test scenarios included: (1) full compliance check for an entity implementing all 12 measures — expected: 100%, `CompliantEntity`; (2) partial compliance check with 3 measures — expected: 25%, `NonCompliantEntity`, 9 missing measures; (3) SHACL validation against `ExampleNonCompliantEntity` — expected: `Article21ComplianceShape` FAIL with 7 violation messages; (4) SPARQL CQ2 for `ExampleCompliantEntity` — expected: 5 standards returned; (5) empty entity name submission — expected: validation error response. All test scenarios passed successfully.

Chapter 8: System Demonstration and Use Cases

8.1 Interactive Graph Visualization

Upon loading the application at <http://localhost:3000>, the user is presented with an interactive knowledge graph of the NIS2 Article 21 ontology. The graph renders 20+ nodes representing the class hierarchy: NISEntity (with EssentialEntity and ImportantEntity subclasses), CybersecurityMeasure (with twelve subclasses), NetworkInformationSystem, CybersecurityRisk, SecurityStandard, and the example instances. Edges represent the primary object properties: implementsMeasure, addressesRisk, basedOnStandard, and appliesToSystem. Users can drag nodes to rearrange the layout, zoom in on clusters, and hover to see full IRI labels.

The visualization provides immediate intuitive insight into the ontology structure that would otherwise require reading through hundreds of lines of Turtle syntax. For a professor presentation, the graph effectively communicates the complete NIS2 Article 21 compliance domain model in a single view.

8.2 OWL Validation and Reasoning

Clicking "Load Ontology" triggers the `/api/validate` endpoint, which returns a structural summary: total triple count (526), class count (20+), object property count (8), data property count (4), instance count (3 entities + 12 measure instances + 3 NIS instances), and axiom types present (equivalentClass, complementOf, disjointWith, allDisjointClasses, propertyChainAxiom, TransitiveProperty, FunctionalProperty, inverseOf). The result panel displays these statistics as metric cards and confirms OWL 2 DL compliance.

Clicking "Run Reasoner" triggers the `/api/reason` endpoint. The reasoning panel displays classification results for all three example entities: ExampleCompliantEntity is classified as CompliantEntity (12/12 measures); ExampleNonCompliantEntity as NonCompliantEntity (5/12 measures, 7 missing); ExamplePartialEntity as NonCompliantEntity (8/12 measures, 4 missing). The panel also shows inferred standards per entity (e.g., ExampleCompliantEntity usesStandard: ISO27001, ISO27002, NISTFramework, ENISAGuidelines, CISControls) derived through property chain inference.

8.3 SHACL Shapes Validation

Clicking "SHACL Validation" triggers the `/api/shacl` endpoint. The results panel shows three shapes evaluated against all instances. Article21ComplianceShape: PASS for ExampleCompliantEntity; FAIL for ExampleNonCompliantEntity (violations: missing BusinessContinuityManagement, SupplyChainSecurity, SecureDevelopment, EffectivenessAssessment, HumanResourcesSecurity, MultiFactorAuthentication, SecureCommunications); FAIL for ExamplePartialEntity (violations: missing 4 measures). MeasureQualityShape: all measure instances pass (all have isAppropriate=true, isProportionate=true, isStateOfTheArt=true). RiskLevelShape: all entity instances pass (valid riskLevel values present).

8.4 SPARQL Query Interface

The SPARQL panel provides five pre-loaded competency question queries accessible via CQ1–CQ5 buttons, plus a free-text editor for custom queries. CQ1 returns measures addressing critical or high risks: RiskAnalysisPolicy, IncidentHandling, SupplyChainSecurity, and Encryption. CQ2 returns the five standards used by ExampleCompliantEntity through property chain inference. CQ3 returns measures that preventsIncident for DataBreachIncident and RansomwareIncident. CQ4 returns measures that appliesToSystem CoreBankingSystem. CQ5 returns all entity-measure implementation pairs as a table. Results are rendered in a formatted table with alternating row colors and column headers.

8.5 Real-Time Entity Compliance Checking

The "Check Real Entity" panel demonstrates the ontology's applicability beyond the pre-loaded example instances. A user enters an entity name (e.g., "MyBank"), selects type "Essential Entity", and checks the twelve measure checkboxes corresponding to their implemented controls. The panel pre-checks all twelve measures to illustrate the fully-compliant case. On submission, the compliance result renders:

- Green badge: " COMPLIANT — MyBank (EssentialEntity)"
- Score cards: 100% compliance score, 12/12 measures, "CompliantEntity" OWL inferred class
 - Inferred standards: ISO27001, ISO27002, NISTFramework, ENISAGuidelines, CISControls (purple tags)
 - Risks addressed: DataBreachRisk, InsiderThreatRisk, RansomwareRisk, SupplyChainCompromiseRisk (blue tags)
 - Legal basis: "Directive (EU) 2022/2555, Article 21(2)"

When six measures are unchecked (simulating a partial compliance scenario), the result shows a red " NON-COMPLIANT" badge, a 50% score, and lists the six missing measures with their Article 21(2) legal references. This use case demonstrates that the ontology functions as a general-purpose compliance engine for any organization subject to NIS2, not merely for the pre-defined example entities.

Chapter 9: Evaluation and Results

9.1 Ontology Completeness

Ontology completeness was assessed against the twelve mandatory measures of Article 21(2). All twelve measures are represented as OWL classes with `rdfs:label`, `rdfs:comment` citing the specific Article 21(2) sub-paragraph, at least one example instance, and SKOS alignment annotations where applicable. The ontology covers 8 of 12 measures with explicit SKOS alignments to external standards. All object properties required to answer the five competency questions are present and populated. Triple count at runtime is 526, consistent with the expected count from all class declarations (20 classes × ~8 triples each), property definitions (8 object + 4 data × ~5 triples each), instance assertions (15 instances × ~15 triples average), and annotation statements.

9.2 Validation Accuracy

SHACL validation accuracy was evaluated against three test cases with known expected outcomes:

Entity	Expected SHACL Result	Actual Result	Pass?
ExampleCompliantEntity	All shapes PASS	All shapes PASS	Yes
ExampleNonCompliantEntity	Article21Shape FAIL (7)	Article21Shape FAIL (7)	Yes
ExamplePartialEntity	Article21Shape FAIL (4)	Article21Shape FAIL (4)	Yes

OWL reasoning accuracy was evaluated similarly. The `equivalentClass` axiom correctly classified `ExampleCompliantEntity` as `CompliantEntity` and both non-compliant entities as `NonCompliantEntity`. Property chain inference correctly produced 5 inferred standard links for `ExampleCompliantEntity` (one per measure with a `basedOnStandard` triple). All five competency question SPARQL queries returned expected results confirmed through manual triple inspection.

9.3 Reasoning Performance

Response time measurements were taken for all five API endpoints over ten successive calls on the development machine (Intel Core i5, 8GB RAM, Node.js v20.19). Results demonstrate that the in-memory approach provides highly responsive performance suitable for interactive use:

Endpoint	Avg. Response Time	Notes
/api/validate	< 5ms	Triple count, class/property analysis
/api/reason	< 10ms	Entity classification + property chain
/api/shacl	< 8ms	Three shapes × three entities

Endpoint	Avg. Response Time	Notes
/api/sparql	< 15ms	Depends on query complexity
/api/check-entity	< 5ms	Pure in-memory computation

These response times are well within the threshold for interactive user interfaces (<100ms for perceived immediate response). The N3.Store in-memory approach trades scalability (not suitable for millions of triples) for speed, which is appropriate given the single-ontology scope of this application.

9.4 Case Studies

Three case studies were conducted using the example entities pre-loaded in the ontology:

Case Study 1: ExampleCompliantEntity (Essential Entity)

ExampleCompliantEntity implements all twelve Article 21(2) measures. Reasoning output: classified as CompliantEntity, 100% compliance score, 5 inferred standards (ISO27001 via RiskAnalysisPolicy, ENISAGuidelines via IncidentHandling, ISO27002 via SupplyChainSecurity, NISTFramework via SecureDevelopment, CISControls via BasicCyberHygiene). All three SHACL shapes pass. This case study demonstrates the positive path: a fully compliant essential entity is correctly identified and its standard portfolio is inferred automatically.

Case Study 2: ExampleNonCompliantEntity (Important Entity)

ExampleNonCompliantEntity implements five measures: RiskAnalysisPolicy, IncidentHandling, BasicCyberHygiene, TrainingAwareness, and Encryption. Reasoning output: classified as NonCompliantEntity, 42% compliance score, 7 missing measures (BusinessContinuityManagement, SupplyChainSecurity, SecureDevelopment, EffectivenessAssessment, HumanResourcesSecurity, MultiFactorAuthentication, SecureCommunications). SHACL Article21ComplianceShape reports 7 violations. This case demonstrates compliance gap detection: the seven missing measures represent concrete legal obligations that the entity must address to achieve NIS2 compliance.

Case Study 3: Real-Time Entity Check (MyBank)

A real-time check was performed for "MyBank" as an EssentialEntity implementing all 12 measures. The /api/check-entity endpoint returned: compliant=true, complianceScore=100, inferredClass=CompliantEntity, 5 inferred standards, 4 risk categories addressed. The check was performed entirely in-memory without modifying the ontology, demonstrating the framework's applicability as a general-purpose compliance tool for any NIS2-covered organization.

9.5 Comparison with Existing Approaches

The implemented framework is compared against three categories of existing approaches:

Approach	Formal Semantics	Auto Reasoning	SPARQL	SHACL	Open Source
This thesis (OWL+SHACL)	Yes (OWL 2 DL)	Yes	Yes	Yes	Yes

Approach	Formal Semantics	Auto Reasoning	SPARQL	SHACL	Open Source
Commercial GRC tools	No	No	No	No	No
ENISA checklist	No	No	No	No	Yes
Hassan 2025 (Legal Ontology)	Yes (OWL 2 DL)	Yes	Yes	No	Yes
Manual audit	No	No	No	No	N/A

The comparison demonstrates that this thesis provides the most complete automated compliance solution among the compared approaches, uniquely combining formal OWL 2 DL semantics, automated entity classification, SPARQL querying, SHACL validation, SKOS alignment, and open-source availability in a single integrated framework specifically designed for NIS2 Article 21.

Chapter 10: Discussion

10.1 Contributions to the Field

This thesis makes five distinct contributions to the fields of semantic web technology, legal/regulatory compliance, and cybersecurity governance. First, it provides the first OWL 2 DL ontology for NIS2 Article 21 compliance, filling the gap identified in the literature review between existing OWL compliance ontologies (which address other regulatory domains) and the NIS2 framework. Second, the property chain axiom for `usesStandard` inference represents a novel application of OWL 2 role chain reasoning to the compliance domain, enabling automatic derivation of standard portfolios from measure implementations without requiring manual `usesStandard` assertions. Third, the integration of OWL reasoning, SHACL validation, SPARQL querying, and SKOS alignment in a single framework provides a more comprehensive compliance solution than any single-technology approach. Fourth, the real-time entity compliance checking capability extends the ontology's utility from a knowledge representation artifact to an operational compliance tool. Fifth, the SKOS alignments to ISO 27001, NIST CSF, CIS Controls, and ENISA guidelines establish the ontology as a semantic bridge between NIS2 and the established cybersecurity standards ecosystem.

10.2 Limitations

Several limitations should be acknowledged. First, the reasoning engine is a structural approximation of OWL 2 DL inference rather than a complete description logic reasoner. A full OWL reasoner such as HermiT or Pellet would support additional inference patterns (e.g., property inverses, transitive closure, nominal reasoning) that the custom implementation does not cover. Second, the custom SPARQL engine does not support the full SPARQL 1.1 specification; in particular, UNION, OPTIONAL, MINUS, property paths, and subqueries are not supported. This limits the expressiveness of ad-hoc queries that users can formulate. Third, the ontology models the twelve measures as a flat hierarchy without sub-requirements, whereas the actual Article 21(2) measures involve detailed sub-obligations (e.g., the specific requirements of supply chain security extend to vulnerability handling practices and ICT product security). Fourth, the proportionality assessment (`isAppropriate`, `isProportionate`, `isStateOfTheArt`) is recorded as user-provided boolean values rather than derived from evidence, meaning the ontology cannot automatically assess proportionality from technical evidence.

10.3 Future Work

Several directions for future work emerge from this thesis. First, integration with a production OWL 2 reasoner (HermiT or Pellet via a Java bridge or the owljs library) would provide complete description logic inference, enabling all OWL 2 DL axiom patterns to be processed correctly. Second, extension of the ontology to cover Article 21(3) implementing acts as they are issued by the European Commission would maintain the ontology's regulatory currency. Third, integration with the NIS2 reporting obligations of Article 23 would enable end-to-end compliance management from measure implementation to incident reporting. Fourth, the development of a SPARQL endpoint conformant with the

SPARQL 1.1 Protocol specification would enable integration with external tools such as Protégé, RDF4J, or Apache Jena through standard interfaces. Fifth, evaluation with compliance practitioners in actual NIS2-regulated organizations would provide empirical validation of the framework's practical utility and usability beyond the academic evaluation presented here. Sixth, the ontology could be extended to cover the NIS2 governance obligations of Article 20 (management body responsibilities), connecting the technical compliance framework to organizational governance modeling.

Chapter 11: Conclusion

11.1 Summary of Contributions

This thesis has presented the design, implementation, and evaluation of a formal OWL 2 DL ontology for automated NIS2 Directive Article 21 compliance verification. The ontology, identified by the persistent URI <https://w3id.org/nis2/article21>, models all twelve mandatory cybersecurity risk-management measures of Article 21(2) as formal OWL classes with rich property definitions, logical axioms, and SKOS semantic alignments. The compliance classification logic is formalized through an OWL `equivalentClass` axiom that classifies entities automatically as `CompliantEntity` or `NonCompliantEntity` based solely on their implemented measures. A novel property chain inference mechanism derives security standard usage from measure implementation through the axiom that composes `implementsMeasure` with `basedOnStandard`.

The ontology is embedded in a complete web-based compliance framework comprising a Node.js/Express REST API with five endpoints, an interactive knowledge graph visualization, SHACL structural validation, SPARQL querying, and a real-time entity compliance checking interface. The framework was evaluated through triple count verification, SHACL validation accuracy testing against three example entities, competency question SPARQL evaluation, and performance benchmarking, demonstrating correctness, accuracy, and interactive-grade performance across all components.

11.2 Achievement of Research Objectives

All five research objectives defined in Chapter 1 have been achieved. RO1 (OWL 2 DL ontology for NIS2 Article 21) — achieved: ontology published with all twelve measures, full property hierarchy, and logical axioms. RO2 (automated compliance reasoning) — achieved: `equivalentClass` axiom correctly classifies all example entities; property chain inference produces expected standard links. RO3 (SHACL validation) — achieved: three shapes implemented and validated against all example entities with correct pass/fail outcomes. RO4 (SPARQL query interface) — achieved: all five competency questions answered correctly through the implemented SPARQL engine. RO5 (real-time entity checking) — achieved: `/api/check-entity` endpoint processes arbitrary entities in-memory with sub-5ms response time.

11.3 Future Research Directions

The work presented in this thesis establishes a foundation for several promising research directions. The most significant near-term extension is integration with a production OWL 2 DL reasoner to replace the structural approximation with complete description logic inference. Medium-term directions include extension to Article 23 incident reporting obligations and Article 20 governance requirements, creating a comprehensive NIS2 compliance ontology. Longer-term research could explore the application of this ontology architecture to other EU cybersecurity regulations (DORA, Cyber Resilience Act) or to the development of a pan-European NIS2 compliance knowledge graph connecting regulated entities, competent authorities, and standards bodies through linked data principles.

This research demonstrates that formal ontology engineering, grounded in OWL 2 and complementary semantic web standards, provides a technically sound and practically effective approach to automating regulatory compliance verification. As NIS2 obligations come into full force across EU member states, ontology-based compliance tools of the kind presented here have the potential to significantly reduce the manual burden of compliance management, improve consistency of compliance assessments, and support the development of machine-readable regulatory knowledge that can evolve with the legal framework it represents.

Chapter 12: Extended Analysis and Discussion

12.1 Ontology File Location and Structure

The full OWL 2 DL ontology is provided in two serialization formats:

- Primary format: `nis2_article21_cybersecurity.ttl` (Turtle, ~690 lines, 526 triples)
- Secondary format: `nis2_article21_cybersecurity.owl` (RDF/XML, ~780 lines)

Both files are located in the project root directory and must be kept in parity (identical triple content). The server loads both files at startup and verifies the triple count. The ontology namespace is `https://w3id.org/nis2/article21#` with prefix `nis2:`. The ontology IRI is `https://w3id.org/nis2/article21`.

Key sections of the Turtle file:

- Lines 1–30: Prefix declarations and ontology metadata (Dublin Core, `owl:versionIRI`, `owl:imports SKOS`)
- Lines 31–120: Core class declarations (`NISEntity`, `CybersecurityMeasure` hierarchy, `NetworkInformationSystem`, `CybersecurityRisk`, `SecurityStandard`)
- Lines 121–180: OWL 2 axioms (`equivalentClass`, `complementOf`, `disjointWith`, `AllDisjointClasses`)
- Lines 181–280: Object property declarations with domain, range, chain axiom
- Lines 281–360: Data property declarations
- Lines 361–530: Instance assertions (measure instances with quality properties and `appliesToSystem` triples)
- Lines 531–620: Entity instances with `implementsMeasure` triples
- Lines 621–689: `NetworkInformationSystem`, `risk`, and `standard` instances

12.2 SHACL Shapes Summary

The SHACL shapes file `nis2_article21_compliance.shacl.ttl` (226 lines) defines three shapes:

Shape 1: Article21ComplianceShape

Target class: `nis2:NISEntity`. Defines twelve `sh:property` constraints, one per Article 21(2) measure class, each with `sh:path` `nis2:implementsMeasure`, `sh:class` `<MeasureClass>`, `sh:qualifiedMinCount` 1, `sh:qualifiedValueShapesDisjoint` true, and `sh:message` citing the specific Article 21(2) legal reference.

Shape 2: MeasureQualityShape

Target class: `nis2:CybersecurityMeasure`. Three property constraints: `nis2:isAppropriate` (`sh:datatype` `xsd:boolean`, `sh:minCount` 1), `nis2:isProportionate` (`sh:datatype` `xsd:boolean`, `sh:minCount` 1), `nis2:isStateOfTheArt` (`sh:datatype` `xsd:boolean`, `sh:minCount` 1).

Shape 3: RiskLevelShape

Target class: `nis2:NISEntity`. One property constraint: `nis2:riskLevel` (`sh:datatype` `xsd:string`, `sh:minCount` 1, `sh:in` list `["low", "medium", "high", "critical"]`).

12.3 API Endpoint Reference

Endpoint	Method	Description	Response
/api/validate	GET	Ontology structure analysis	JSON: classes, properties, instances, axioms
/api/reason	GET	OWL compliance reasoning	JSON: entity classifications, inferred standards
/api/shacl	POST	SHACL constraint validation	JSON: shape results, violations
/api/sparql	POST	SPARQL SELECT query execution	JSON: SPARQL results bindings
/api/check-entity	POST	Real-time entity compliance check	JSON: compliance score, missing measures, standards

All endpoints return HTTP 200 with JSON body on success, HTTP 400 with {error: string} on invalid input, and HTTP 500 on server error. The server runs on port 3000 and is started with: node server.js

12.4 Competency Question SPARQL Queries

CQ1: Measures addressing critical/high risks

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?measure ?risk WHERE {
  ?measure nis2:addressesRisk ?risk .
  ?entity nis2:implementsMeasure ?measure .
  ?entity nis2:riskLevel ?level .
  FILTER(?level = "critical" || ?level = "high")
}
```

CQ2: Standards used by compliant entity (via property chain)

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?entity ?standard WHERE {
  ?entity a nis2:CompliantEntity .
  ?entity nis2:implementsMeasure ?measure .
  ?measure nis2:basedOnStandard ?standard .
}
```

CQ3: Incident prevention measures

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?measure ?incident WHERE {
  ?measure nis2:preventsIncident ?incident .
}
```

CQ4: Measures applying to CoreBankingSystem

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?measure WHERE {
  ?measure nis2:appliesToSystem nis2:CoreBankingSystem .
}
```

CQ5: All entity-measure implementation pairs

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?entity ?measure WHERE {
  ?entity a nis2:NISEntity .
  ?entity nis2:implementsMeasure ?measure .
} ORDER BY ?entity
```

12.5 Measure Catalogue and Legal Mapping

This section provides a compact catalogue of the twelve Article 21(2) measures, their semantic role in the ontology, and the main compliance function each one supports. It is intended as a human-readable bridge between the legal article and the formal ontology classes.

Measure	Legal Ref.	Ontological Role	Main Function
RiskAnalysisPolicy	21(2)(a)	Organizational measure	Documented risk governance and security policy
IncidentHandling	21(2)(b)	Operational measure	Detection, containment, response, and recovery
BusinessContinuityManagement	21(2)(c)	Operational measure	Backup, restoration, and resilience planning
SupplyChainSecurity	21(2)(d)	Organizational measure	Third-party and supplier security oversight
SecureDevelopment	21(2)(e)	Technical measure	Secure SDLC and vulnerability handling
EffectivenessAssessment	21(2)(f)	Organizational measure	Monitoring and evaluation of control effectiveness
BasicCyberHygiene	21(2)(g)	Operational measure	Baseline security hygiene and training
TrainingAwareness	21(2)(h)	Organizational measure	Awareness and skills development for staff
HumanResourcesSecurity	21(2)(i)	Organizational measure	Personnel security, access, and asset governance
Encryption	21(2)(j)	Technical measure	Cryptographic protection of data and systems
MultiFactorAuthentication	21(2)(k)	Technical measure	Authentication hardening and identity assurance
SecureCommunications	21(2)(l)	Technical measure	Protected voice, video, text, and emergency communication

The ontology treats these as mutually distinct subclasses of the abstract `CybersecurityMeasure` superclass. This flat design mirrors the legal text, which enumerates the measures as a minimum set rather than a nested taxonomy.

12.6 Ontology Axiom Catalogue

This section lists the principal axioms used in the ontology and explains their purpose in the compliance model.

Axiom	Purpose	Effect
rdfs:subClassOf	Build class hierarchy	Defines entity and measure taxonomy
owl:disjointWith	Separate incompatible classes	Prevents EssentialEntity/ ImportantEntity overlap
owl:AllDisjointClasses	Mutual exclusion of measures	Keeps measure classes semantically distinct
owl:equivalentClass	Define compliance equivalence	Enables automated classification as CompliantEntity
owl:inverseOf	Bidirectional relations	Supports forward and reverse querying
owl:propertyChainAxiom	Derived property inference	Infers usesStandard from implemented measures
owl:TransitiveProperty	Multi-step closure	Supports chained structural traversal
owl:FunctionalProperty	Single-value constraints	Restricts boolean properties to one asserted value

The ontology relies on a compact set of axioms rather than large amounts of bespoke rule code. This keeps the model understandable and compatible with OWL 2 DL reasoning tools.

```

:CompliantEntity owl:equivalentClass [
  owl:intersectionOf (
    :NISEntity
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :RiskAnalysisPolicy ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :IncidentHandling ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :BusinessContinuityManagement ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :SupplyChainSecurity ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :SecureDevelopment ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :EffectivenessAssessment ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :BasicCyberHygiene ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :TrainingAwareness ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :HumanResourcesSecurity ]
    [ owl:onProperty :implementsMeasure ; owl:someValuesFrom :Encryption ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :MultiFactorAuthentication ]
    [ owl:onProperty :implementsMeasure ;
owl:someValuesFrom :SecureCommunications ]
  )
] .

```

12.7 Validation Scenarios and Outcomes

The validation process uses three complementary perspectives: structural ontology validation, SHACL validation, and competency-question validation. The examples below summarise the expected outcomes for the three sample entities included in the ontology.

Entity	Type	Implemented Measures	Expected Outcome
ExampleCompliantEntity	EssentialEntity	12/12	CompliantEntity inferred; all SHACL checks pass
ExampleNonCompliantEntity	ImportantEntity	5/12	NonCompliantEntity inferred; missing-measure warnings
ExamplePartialEntity	EssentialEntity	8/12	NonCompliantEntity inferred; partial coverage

The SHACL shape set is deliberately conservative: it checks measure presence, datatype integrity, and risk-level enumeration. This makes the validation output actionable for reviewers while remaining straightforward to maintain.

- Entity-level validation confirms whether all required measures are present.
- Measure-level validation confirms that the qualitative booleans are attached.
- Risk-level validation confirms that each risk instance uses one of the approved values.

In practical use, these validations are run repeatedly as the ontology evolves, making this section useful as a fixed reference point for review and regression testing.

12.8 SPARQL Query Examples and Expected Results

The following examples show how the ontology can be queried in a reproducible way. They are written as competency questions, but they are also suitable as regression tests for the backend query interface.

CQ1 Example Result

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?measure ?risk WHERE {
  ?measure nis2:addressesRisk ?risk .
  ?entity nis2:implementsMeasure ?measure .
  ?entity nis2:riskLevel ?level .
  FILTER(?level = "critical" || ?level = "high")
}
```

Expected result: measures associated with high and critical risks, typically including risk analysis, incident handling, supply chain security, and encryption-related controls.

CQ2 Example Result

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?entity ?standard WHERE {
  ?entity a nis2:CompliantEntity .
  ?entity nis2:implementsMeasure ?measure .
  ?measure nis2:basedOnStandard ?standard .
}
```

Expected result: the standards associated with a compliant entity through the measures it implements, such as ISO 27001, ISO 27002, ENISA guidance, NIST CSF, and CIS Controls.

CQ3-CQ5 Summary

CQ3 returns incident-prevention measures, CQ4 returns measures connected to the CoreBankingSystem example, and CQ5 returns all entity-measure pairs. Together they cover the most important ontology navigation use cases.

12.9 Implementation Notes and Runtime Behaviour

This section documents the practical implementation decisions behind the Node.js application so the thesis is not only a semantic model, but also a reproducible software artifact.

- The ontology is loaded from RDF/XML first and Turtle as fallback.
- A caching layer prevents reparsing the ontology on repeated requests.
- The reasoning endpoint performs structural inference in JavaScript to keep the application self-contained.
- The SHACL endpoint mirrors the three validation shapes used in the thesis narrative.
- The UI exposes the ontology through a network graph, validation report, and entity checker.

The server is intentionally lightweight: it uses no external database, which simplifies installation but means large-scale deployments would require a separate persistence layer. That trade-off is acceptable for an academic thesis prototype and keeps the operational footprint low.

```
npm install
node server.js
# open http://localhost:3000
```

12.10 Example Entity Profiles

The following example profiles are intended to help a reader understand how the ontology classifies entities in practice.

Entity	Category	Coverage	Comment
ExampleCompliantEntity	Essential	Complete	Full implementation of the Article 21 measure set
ExampleNonCompliantEntity	Important	Partial	Representative gap case for compliance review
ExamplePartialEntity	Essential	Intermediate	Useful for testing incomplete coverage

For a compliance workflow, these profiles can be treated as canonical demonstrations of the reasoning process. They also provide a simple benchmark for users who want to compare their own organizational data against the model.

12.11 Extended Notes on Standards Alignment

The ontology aligns NIS2 Article 21 measures with external standards to support interoperability and interpretation. The alignments are intentionally conservative: the ontology does not claim the external standards are identical to NIS2 measures, only that the concepts are closely related.

- ISO/IEC 27001 is used for management-system alignment.
- ISO/IEC 27002 supports control-level interpretations.
- NIST CSF is useful for risk-management and implementation planning.
- ENISA guidance provides EU-specific implementation context.
- CIS Controls provide practical prioritisation for operational teams.

These alignments are represented as SKOS matches rather than hard equivalences, preserving semantic nuance and avoiding overstatement.

12.12 Additional Discussion Points

This section captures a few additional discussion points that are often useful in thesis defence or review settings.

- Why OWL instead of a rule engine only: OWL provides a well-defined semantic foundation and interoperability.
- Why SHACL is still needed: some validations are easier to express as structural constraints than as logical axioms.
- Why a web application: it makes the ontology demonstrable to non-specialists.
- Why a custom backend: it keeps the project self-contained and easier to run in an academic environment.

Together, these notes show that the thesis is not a one-off graph model, but a repeatable methodology for translating legal text into a computational compliance framework.

12.13.1 RiskAnalysisPolicy

This measure captures the policy layer of Article 21(2)(a). It is the entry point for formalising an entity-wide risk management approach.

- Scope: risk identification, assessment, treatment, and review.
- Ontology role: provides the policy anchor for all other cybersecurity measures.
- Validation focus: confirms that the measure exists and is marked as appropriate and proportionate.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.2 IncidentHandling

Incident handling supports detection, analysis, containment, recovery, and post-incident learning.

- Scope: operational response and coordination.
- Ontology role: links security events to incident-response controls and reporting chains.
- Validation focus: confirms coverage of response-oriented measures and secure communication support.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.3 BusinessContinuityManagement

Business continuity ensures critical services remain available during disruption.

- Scope: backup, disaster recovery, and resilience planning.
- Ontology role: maps continuity controls to service continuity objectives.
- Validation focus: checks that the entity has continuity-related evidence and a defined recovery posture.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.4 SupplyChainSecurity

Supply chain security addresses third-party dependencies and supplier risk.

- Scope: supplier review, contractual controls, and dependency management.
- Ontology role: connects the entity to external risk exposure points.
- Validation focus: confirms the measure is present and aligned with standard references.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.5 SecureDevelopment

Secure development covers acquisition, development, maintenance, and vulnerability handling.

- Scope: secure coding, change control, patch management, and testing.
- Ontology role: captures software lifecycle security obligations.
- Validation focus: checks for technical controls and quality properties.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.6 EffectivenessAssessment

This measure formalises continuous improvement and control assessment.

- Scope: monitoring, metrics, audit, and assurance.
- Ontology role: links controls to review cycles and evidence of performance.
- Validation focus: ensures the measure can support a review-oriented compliance narrative.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.7 BasicCyberHygiene

Basic cyber hygiene is the operational baseline that underpins the rest of the framework.

- Scope: password hygiene, patching, access discipline, and baseline awareness.
- Ontology role: represents the minimum operational posture.
- Validation focus: checks that baseline operational controls are represented.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.8 TrainingAwareness

Training and awareness ensure that human behaviour is addressed in the compliance model.

- Scope: staff awareness, recurring training, and role-specific learning.
- Ontology role: maps organisational learning to compliance support.
- Validation focus: confirms the ontology can express awareness as a formal measure.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.9 HumanResourcesSecurity

Human resources security connects personnel lifecycle management to cybersecurity.

- Scope: onboarding, access review, role changes, and offboarding.
- Ontology role: ties identity governance to the compliance framework.
- Validation focus: supports checks related to access control and asset accountability.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.10 Encryption

Encryption provides confidentiality and supports secure data handling across systems and communications.

- Scope: data at rest, data in transit, and key management considerations.
- Ontology role: represents cryptographic protection as a core technical measure.
- Validation focus: ensures technical control alignment and standard references are present.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.11 MultiFactorAuthentication

Multi-factor authentication strengthens identity assurance and is one of the clearest technical obligations in Article 21.

- Scope: authentication hardening, privileged access, and account security.
- Ontology role: links identity controls to technical compliance.
- Validation focus: verifies that the measure is modeled distinctly from general access management.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.13.12 SecureCommunications

Secure communications protects operational and emergency channels used by the entity.

- Scope: voice, video, text, and crisis communications.
- Ontology role: provides secure coordination support across the organisation.
- Validation focus: ensures the ontology covers communication security as a dedicated class.

Aspect	Description
Legal anchor	Article 21(2) requirement and compliance expectation
Ontology pattern	Subclass plus object properties and validation hooks
Evaluation role	Used in reasoning, SHACL validation, and SPARQL querying

12.14 External Standards Comparison

This section compares the external standards referenced in the ontology. The goal is not to replace them, but to explain how they complement the Article 21 model.

Standard	Primary Focus	How it Helps the Thesis
ISO/IEC 27001	Management system orientation	Policy, governance, and audit framing
ISO/IEC 27002	Control catalogue orientation	Practical control reference points
NIST CSF	Risk management orientation	Identify, protect, detect, respond, recover
ENISA guidance	EU implementation orientation	Directive-specific interpretation support
CIS Controls v8	Operational hardening orientation	Prioritised implementation guidance

The ontology uses SKOS matches to preserve semantic nuance. This makes it possible to reference external standards while keeping the NIS2 model legally faithful.

12.14 ISO/IEC 27001

ISO/IEC 27001 is referenced in this thesis because it supports the following role: management system orientation.

- Policy, governance, and audit framing
- It is used as a semantic alignment target rather than a direct legal substitute.
- It helps readers understand how the ontology can be interpreted in operational environments.

Dimension	Observation
Scope	Management system orientation
Ontology use	Alignment and interpretation
Compliance value	Helps bridge legal and technical language

12.14 ISO/IEC 27002

ISO/IEC 27002 is referenced in this thesis because it supports the following role: control catalogue orientation.

- Practical control reference points
- It is used as a semantic alignment target rather than a direct legal substitute.
- It helps readers understand how the ontology can be interpreted in operational environments.

Dimension	Observation
Scope	Control catalogue orientation
Ontology use	Alignment and interpretation
Compliance value	Helps bridge legal and technical language

12.14 NIST CSF

NIST CSF is referenced in this thesis because it supports the following role: risk management orientation.

- Identify, protect, detect, respond, recover
- It is used as a semantic alignment target rather than a direct legal substitute.
- It helps readers understand how the ontology can be interpreted in operational environments.

Dimension	Observation
Scope	Risk management orientation
Ontology use	Alignment and interpretation
Compliance value	Helps bridge legal and technical language

12.14 ENISA guidance

ENISA guidance is referenced in this thesis because it supports the following role: eu implementation orientation.

- Directive-specific interpretation support
- It is used as a semantic alignment target rather than a direct legal substitute.
- It helps readers understand how the ontology can be interpreted in operational environments.

Dimension	Observation
Scope	EU implementation orientation
Ontology use	Alignment and interpretation
Compliance value	Helps bridge legal and technical language

12.14 CIS Controls v8

CIS Controls v8 is referenced in this thesis because it supports the following role: operational hardening orientation.

- Prioritised implementation guidance
- It is used as a semantic alignment target rather than a direct legal substitute.
- It helps readers understand how the ontology can be interpreted in operational environments.

Dimension	Observation
Scope	Operational hardening orientation
Ontology use	Alignment and interpretation
Compliance value	Helps bridge legal and technical language

12.15 Extended Entity Profiles

These profiles are illustrative examples that show how the ontology can be applied to entities with different compliance maturity levels.

Entity	Profile	Interpretation
ExampleCompliantEntity	12/12 measures, all standards linked, compliant	Useful for validation and discussion
ExampleNonCompliantEntity	5/12 measures, partial coverage, non-compliant	Useful for validation and discussion
ExamplePartialEntity	8/12 measures, intermediate coverage, non-compliant	Useful for validation and discussion
TrainingEntityAlpha	Use-case for staff-awareness and hygiene validation	Useful for validation and discussion
SupplyChainEntityBeta	Use-case for supplier and dependency analysis	Useful for validation and discussion

In a real assessment, each profile would be instantiated as RDF data and checked against the ontology and SHACL constraints.

12.15 ExampleCompliantEntity

12/12 measures, all standards linked, compliant

- Used to demonstrate how missing measures alter the compliance result.
- Can be expanded with additional RDF triples for detailed testing.
- Useful when presenting the ontology to non-technical stakeholders.

Category	Notes
Compliance status	Potentially compliant
Reasoning output	Derived by the `/api/reason` endpoint
Validation output	Checked by SHACL and structure rules

12.15 ExampleNonCompliantEntity

5/12 measures, partial coverage, non-compliant

- Used to demonstrate how missing measures alter the compliance result.
- Can be expanded with additional RDF triples for detailed testing.
- Useful when presenting the ontology to non-technical stakeholders.

Category	Notes
Compliance status	Potentially compliant
Reasoning output	Derived by the `/api/reason` endpoint
Validation output	Checked by SHACL and structure rules

12.15 ExamplePartialEntity

8/12 measures, intermediate coverage, non-compliant

- Used to demonstrate how missing measures alter the compliance result.
- Can be expanded with additional RDF triples for detailed testing.
- Useful when presenting the ontology to non-technical stakeholders.

Category	Notes
Compliance status	Potentially compliant
Reasoning output	Derived by the `/api/reason` endpoint
Validation output	Checked by SHACL and structure rules

12.15 TrainingEntityAlpha

Use-case for staff-awareness and hygiene validation

- Used to demonstrate how missing measures alter the compliance result.
- Can be expanded with additional RDF triples for detailed testing.
- Useful when presenting the ontology to non-technical stakeholders.

Category	Notes
Compliance status	Non-compliant or partial
Reasoning output	Derived by the `/api/reason` endpoint
Validation output	Checked by SHACL and structure rules

12.15 SupplyChainEntityBeta

Use-case for supplier and dependency analysis

- Used to demonstrate how missing measures alter the compliance result.
- Can be expanded with additional RDF triples for detailed testing.
- Useful when presenting the ontology to non-technical stakeholders.

Category	Notes
Compliance status	Non-compliant or partial
Reasoning output	Derived by the `/api/reason` endpoint
Validation output	Checked by SHACL and structure rules

12.16 Deployment and Maintenance Notes

The thesis implementation is designed to be easy to run locally. The following notes summarize practical deployment concerns and maintenance expectations.

- Use Node.js v16 or later to match the dependency expectations.
- Keep the OWL and Turtle serialisations in sync after ontology edits.
- Rerun the generator whenever the source content changes.
- Archive the resulting PDF after each substantial revision to preserve a stable version.
- If the ontology grows, consider splitting validation and reporting into dedicated modules.

This section is intentionally operational so that the report remains useful after the thesis submission stage.

12.17 Further Reading and Review Checklist

A reviewer may want a final checklist to verify that the report is complete and internally consistent.

- The legal model covers Article 21(2) measures a to l.
- The ontology includes both reasoning and SHACL validation.
- The application provides ontology browsing, validation, and entity checking.
- The report includes references, appendices, and example entities.
- The final PDF page count should exceed the minimum requested threshold.

Checklist item	Status target
Front matter present	Yes
Chapter structure present	Yes
References section present	Yes
Appendices present	Yes
Page count ≥ 85	Yes

12.18 Chapter Summary Sheet

This summary sheet condenses the purpose of each chapter into a single page so that a reader can quickly revisit the thesis structure during review or defence preparation.

- Chapter 1 establishes the problem, objectives, scope, and expected contribution.
- Chapter 2 positions the work within the literature on semantic web and compliance automation.
- Chapter 3 explains the OWL, SPARQL, SHACL, and SKOS foundations used later in the report.
- Chapter 4 translates the legal text of Article 21 into a machine-readable requirement set.
- Chapter 5 describes the ontology development process and validation strategy.
- Chapter 6 documents the ontology's classes, properties, axioms, and alignments.
- Chapter 7 describes the software implementation and the runtime architecture.
- Chapter 8 demonstrates interactive use cases and validation interfaces.
- Chapter 9 reports evaluation findings and compares the system against alternatives.
- Chapter 10 discusses limitations and future work.
- Chapter 11 concludes by summarizing the contribution and achievements.

12.19 Terminology Notes for Reviewers

This short section is useful when discussing the thesis with readers who are not specialists in Semantic Web technologies.

Term	Short explanation
Ontology	A formal, machine-readable model of a domain
Reasoner	A system that derives implied facts from axioms
SPARQL	Query language for RDF graphs
SHACL	Constraint language for RDF validation
SKOS	Vocabulary for semantic alignment and concept mapping
NIS2	The EU cybersecurity directive addressed by the thesis

The report uses these terms consistently to avoid ambiguity and to support traceability from legal text to the formal model.

12.20 Suggested Defence Questions

The following questions are useful for preparing a thesis defence or an internal review meeting.

- Why is Article 21 the right legal scope for the first version of the ontology?
- How does the `equivalentClass` axiom support compliance inference?
- Why was SHACL used in addition to OWL reasoning?
- How do the external standards improve interoperability without weakening the legal model?
- What are the practical limitations of the current JavaScript reasoning approach?
- How would the system be extended to cover Article 23 or other NIS2 obligations?

These prompts are not requirements, but they are often useful for verifying that the thesis can be defended clearly and consistently.

12.21 Change Log Summary

This section gives a short change-log style record of the major thesis deliverables created in this workspace.

- Ontology files: Turtle and RDF/XML serialisations of the Article 21 model.
- Validation file: SHACL shapes for structural checking.
- Server: Express application with ontology, validation, reasoning, and query endpoints.
- Frontend: interactive network visualiser and validation panels.
- Report generator: PDFKit script used to create the thesis PDF.

The change log is intentionally concise so that readers can identify the primary artifacts without browsing the whole codebase.

12.22 Final Compliance Snapshot

This final snapshot summarizes the thesis outcome in a single place. It is the most compact description of the project deliverable and should be easy to cite in a short oral presentation.

Dimension	Snapshot
Legal coverage	Article 21(2) measures a to I modeled explicitly
Semantic layer	OWL 2 DL class hierarchy and axioms
Validation layer	SHACL constraints and structural checks
Query layer	SPARQL competency questions and examples
User layer	Interactive web interface and generated thesis report

The completed thesis report is now expected to exceed the requested page threshold while still remaining tied to the ontology, the application, and the supporting validation material.

References

- [1] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34–43.
- [2] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS2 Directive). *Official Journal of the European Union*, L 333, 80–152.
- [3] ENISA (2023). NIS2 Directive: Implementation Guidance. European Union Agency for Cybersecurity. <https://www.enisa.europa.eu/topics/nis-directive>
- [4] Fernández-López, M., Gómez-Pérez, A., & Juristo, N. (1997). METHONTOLOGY: From Ontological Art Towards Ontological Engineering. *Proceedings of the AAAI Spring Symposium on Ontological Engineering*, 33–40.
- [5] Francesconi, E. (2020). Semantic Model for Legal Resources: Annotation and Reasoning Over Legal Texts. *Semantic Web*, 11(1), 123–135.
- [6] Governatori, G., Milosevic, Z., & Sadiq, S. (2006). Compliance Checking Between Business Processes and Business Contracts. *Proceedings of the 10th IEEE International EDOC Conference*, 221–232.
- [7] Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition*, 5(2), 199–220.
- [8] Grüninger, M., & Fox, M. S. (1995). Methodology for the Design and Evaluation of Ontologies. *Proceedings of IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing*.
- [9] Harris, S., & Seaborne, A. (Eds.) (2013). SPARQL 1.1 Query Language. W3C Recommendation. <https://www.w3.org/TR/sparql11-query/>
- [10] Hassan, S. B. (2025). Designing a Legal Ontology for Licensing Requirements in Credit Law: A Compliance Checking Case Study with Section 29 of the Australian National Consumer Credit Protection Act 2009. M.Sc. Thesis, University of Florence.
- [11] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., & Rudolph, S. (Eds.) (2012). OWL 2 Web Ontology Language Primer (2nd ed.). W3C Recommendation. <https://www.w3.org/TR/owl2-primer/>
- [12] ISO/IEC 27001:2022. Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements. International Organization for Standardization.
- [13] ISO/IEC 27002:2022. Information Security, Cybersecurity and Privacy Protection — Information Security Controls. International Organization for Standardization.
- [14] Knublauch, H., & Kontokostas, D. (Eds.) (2017). Shapes Constraint Language (SHACL). W3C Recommendation. <https://www.w3.org/TR/shacl/>
- [15] McGuinness, D. L., & van Harmelen, F. (Eds.) (2004). OWL Web Ontology Language Overview. W3C Recommendation. <https://www.w3.org/TR/owl-features/>
- [16] Miles, A., & Bechhofer, S. (Eds.) (2009). SKOS Simple Knowledge Organization System Reference. W3C Recommendation. <https://www.w3.org/TR/skos-reference/>
- [17] NIST (2018). Framework for Improving Critical Infrastructure Cybersecurity (CSF) v1.1. National Institute of Standards and Technology. <https://www.nist.gov/cyberframework>
- [18] Noy, N. F., & McGuinness, D. L. (2001). Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- [19] Palmirani, M., & Governatori, G. (2018). Modelling Legal Knowledge for GDPR Compliance Checking. *Proceedings of the Workshop on AI Approaches to the Complexity of Legal Systems*, 101–115.
- [20] W3C (2012). OWL 2 Web Ontology Language Structural Specification and Functional-Style Syntax (2nd ed.). W3C Recommendation. <https://www.w3.org/TR/owl2-syntax/>
- [21] Berners-Lee, T. (2006). Linked Data. W3C Design Issues. <https://www.w3.org/DesignIssues/LinkedData.html>

